DB DESIGN THEORY

SCHEMA DECOMPOSITION / NORMALIZATION

Outline

1 Schema decomposition

2 Other dependencies

1. Schema decomposition (1)

- If a relation is not in one of the normal forms, the FDs that cause a violation can give us insight into potential problems.

  The main technique for addressing such problems is *decomposing a relation schema into relation schemas with fewer attributes*.

- **A decomposition of a relation schema** R consists of replacing the relation schema by two (or more) relations that each contain a subset of the attributes of R, and together include all attributes of R.

- We must make sure that each attribute R will appear in at least one relation schema R_i of the decomposition $D = \{R_1, \ldots, R_m\}$ so that no attributes are "lost".

  That is, $\cup_{i=1,m} R_i = R$.

  This is the **attribute preservation property** of the decomposition.

1. Schema decomposition (2)

- *Normal forms when considered in isolation, do not guarantee a good database design*. It is generally not sufficient to check separately that each relation schema in the database is in 3NF or in BCNF.

  The process of normalization through decomposition must also confirm the existence of *additional properties* that the decomposition (the collection of relation schemas) should posses.

- We focus on two properties:

  The **lossless join (LJ) property**, which guarantees that no information is lost by storing only the projections of the instances of the initial relation.

  The **dependency preservation (DP) property**, which ensures that each FD over the initial schema is represented in some individual relation resulting after decomposition.

1.1. DP decomposition (1)

- Intuitively, a dependency preserving decomposition allows us to enforce all FDs *by examining a single relation instance* on each insertion or modification of a tuple (note that deletions cannot cause violation of FDs).

- To define the dependency preserving decomposition formally, we need the concept of a *projection of FDs*. We say that a functional dependency is *applicable* to a schema R if it involves only attributes for R.

- Let R be a relation schema and R_i be a subset of R. Given a set of FDs F on R, the **projection of F on R_i**, denoted, $\Pi_{Ri}(F)$ is the set of dependencies in the closure F^+ (not just F !) that are applicable to R_i.

  That is, a FD $X \rightarrow Y$ in F^+ is in $\Pi_{Ri}(F)$ if the attributes contained in $X \cup Y$ are all contained in R_i.

1.1. DP decomposition (2)

- A decomposition $D = \{R_1, \ldots, R_m\}$ of R is **dependency-preserving w.r.t. F** if the union of the projections of F on each R_i is equivalent to F; that is,

  $$(\Pi_{R1}(F) \cup \ldots \cup \Pi_{Rm}(F))^+ = F^+.$$

  (or using our notation $\Pi_{R1}(F) \cup \ldots \cup \Pi_{Rm}(F) \equiv F$)
- Clearly

  $$(\Pi_{R1}(F) \cup \ldots \cup \Pi_{Rm}(F))^+ \subseteq F^+ \text{ always holds.}$$

  However,

  $$(\Pi_{R1}(F) \cup \ldots \cup \Pi_{Rm}(F))^+ \supseteq F^+ \text{ holds only if the decomposition is dependency preserving.}$$

- The following example shows that it is not sufficient to consider F in checking for dependency preservation. (Instead, we have to consider the closure F^+ of F).

1.1. DP decomposition (3)

- Example:

  Let R = (A, B, C), F = {$A \rightarrow B$, $B \rightarrow C$, $C \rightarrow A$} and d = {AB, BC}

  $A \rightarrow B$ is in $\Pi_{AB}(F)$ and $B \rightarrow C$ is in $\Pi_{BC}(F)$.

  $C \rightarrow A$ is neither in $\Pi_{AB}(F)$ nor in $\Pi_{BC}(F)$, and it *cannot be inferred* from $A \rightarrow B$ and $B \rightarrow C$.

  If we consider F instead of F^+ we will (wrongly) deduce that $C \rightarrow A$ is lost and the decomposition D is not dependency preserving.

  However, observe that from F we can also infer the FDs
  $A \rightarrow C$, $B \rightarrow A$, and $C \rightarrow B$.
  Of these, $B \rightarrow A$ is in $\Pi_{AB}(F)$ and $C \rightarrow B$ is in $\Pi_{BC}(F)$.

  From $B \rightarrow A$ and $C \rightarrow B$ we can infer $C \rightarrow A$.

  Thus the decomposition preserves $C \rightarrow A$, and it is dependency-preserving.

1.2. Problem: loss of information (1)

- Intuitively, when we decompose a relation schema R we want to store the information in any given instance of R *by storing only projections of the instance over the schemas in the decomposition*.

- In general, though, if we take projections of a relation and recombine them using natural join, *we obtain some tuples that were not in the original relation*.

- These tuples are called **spurious tuples**. They result in *loss of information* since we are not anymore able to recover the instance of the original relation from its projections.

1.2. Problem: loss of information (2)

- Example:

S	P	D
s1	p1	d1
s2	p2	d2
s3	p1	d3

Instance r

S	P
s1	p1
s2	p2
s3	p1

$\Pi_{SP}(r)$

P	D
p1	d1
p2	d2
p1	d3

$\Pi_{PD}(r)$

$\Pi_{SP}(r) * \Pi_{PD}(r) =$

S	P	D
s1	p1	d1
s2	p2	d2
s3	p1	d3
s1	p1	d3
s3	p1	d1

By replacing r by $\Pi_{SP}(r)$ and $\Pi_{PD}(r)$ we lose some information: we can no longer say that the facts (s1, p1, d3) and (s3, p1, d1) are not true.

1.2. Problem: loss of information (3)

- Another example:

Consider the schema and instance of relation EMP_PROJ.

EMP_PROJ

SSN	PNUMBER	HOURS	ENAME	PNAME	PLOCATION

EMP_PROJ

SSN	PNUMBER	HOURS	ENAME	PNAME	PLOCATION
123456789	1	32.5	Smith,John B.	ProductX	Bellaire
123456789	2	7.5	Smith,John B.	ProductY	Sugarland
666884444	3	40.0	Narayan,Ramesh K.	ProductZ	Houston
453453453	1	20.0	English,Joyce A.	ProductX	Bellaire
453453453	2	20.0	English,Joyce A.	ProductY	Sugarland
333445555	2	10.0	Wong,Franklin T.	ProductY	Sugarland
333445555	3	10.0	Wong,Franklin T.	ProductZ	Houston
333445555	10	10.0	Wong,Franklin T.	Computerization	Stafford
333445555	20	10.0	Wong,Franklin T.	Reorganization	Houston

1.2. Problem: loss of information (4)

- Another example (cont):

Suppose that we decompose EMP_PROJ into EMP_LOCS and EMP_PROJ1 by projecting EMP_PROJ on the respective attributes .

EMP_LOCS

ENAME	PLOCATION

EMP_PROJ1

SSN	PNUMBER	HOURS	PNAME	PLOCATION

EMP_LOCS

ENAME	PLOCATION
Smith, John B.	Bellaire
Smith, John B.	Sugarland
Narayan, Ramesh K.	Houston
English, Joyce A.	Bellaire
English, Joyce A.	Sugarland
Wong, Franklin T.	Sugarland
Wong, Franklin T.	Houston
Wong, Franklin T.	Stafford

EMP_PROJ1

SSN	PNUMBER	HOURS	PNAME	PLOCATION
123456789	1	32.5	Product X	Bellaire
123456789	2	7.5	Product Y	Sugarland
666884444	3	40.0	Product Z	Houston
453453453	1	20.0	Product X	Bellaire
453453453	2	20.0	Product Y	Sugarland
333445555	2	10.0	Product Y	Sugarland
333445555	3	10.0	Product Z	Houston
333445555	10	10.0	Computerization	Stafford
333445555	20	10.0	Reorganization	Houston

1.2. Problem: loss of information (5)

- Another example (cont):

- If we take the natural join of EMP_PROJ1 and EMP_LOCS the resulting relation contains extra tuples (*spurious tuples*) that were not in the original EMP_PROJ relation (spurious tuples are marked with a * below).

	SSN	PNUMBER	HOURS	PNAME	PLOCATION	ENAME
	123456789	1	32.5	ProductX	Bellaire	Smith,John B.
*	123456789	1	32.5	ProductX	Bellaire	English,Joyce A.
	123456789	2	7.5	ProductY	Sugarland	Smith,John B.
*	123456789	2	7.5	ProductY	Sugarland	English,Joyce A.
*	123456789	2	7.5	ProductY	Sugarland	Wong,Franklin T.
	666884444	3	40.0	ProductZ	Houston	Narayan,Ramesh K.
*	666884444	3	40.0	ProductZ	Houston	Wong,Franklin T.
*	453453453	1	20.0	ProductX	Bellaire	Smith,John B.
	453453453	1	20.0	ProductX	Bellaire	English,Joyce A.
*	453453453	2	20.0	ProductY	Sugarland	Smith,John B.
	453453453	2	20.0	ProductY	Sugarland	English,Joyce A.
*	453453453	2	20.0	ProductY	Sugarland	Wong,Franklin T.
*	333445555	2	10.0	ProductY	Sugarland	Smith,John B.
*	333445555	2	10.0	ProductY	Sugarland	English,Joyce A.
	333445555	2	10.0	ProductY	Sugarland	Wong,Franklin T.
*	333445555	3	10.0	ProductZ	Houston	Narayan,Ramesh K.
	333445555	3	10.0	ProductZ	Houston	Wong,Franklin T.
	333445555	10	10.0	Computerization	Stafford	Wong,Franklin T.
*	333445555	20	10.0	Reorganization	Houston	Narayan,Ramesh K.
	333445555	20	10.0	Reorganization	Houston	Wong,Franklin T.

- We cannot recover the information in EMP_PROJ from EMP_PROJ1 and EMP_LOCS.

1.3. LJ decomposition (1)

- More formally, a decomposition $D = \{R_1, \ldots, R_m\}$ of relation schema R has the **lossless-join property** w.r.t. a set of FDs F if for every instance r of R that satisfies the FDs in F,

  $$\Pi_{R1}(r) * \ldots * \Pi_{Rn}(r) = r$$

- It is easy to see that

  $$\Pi_{R1}(r) * \ldots * \Pi_{Rn}(r) \supseteq r$$

  always holds.
  However, the inclusion

  $$\Pi_{R1}(r) * \ldots * \Pi_{Rn}(r) \subseteq r$$

  holds only if the decomposition is lossless-join.

1.3. LJ decomposition (2)

- **Property** of *binary decompositions*:

  A decomposition $D = \{R_1, R_2\}$ of R is lossless w.r.t. a set of FDs F on R iff either

  - $F \models (R_1 \cap R_2) \rightarrow (R_1 - R_2)$, or
  - $F \models (R_1 \cap R_2) \rightarrow (R_2 - R_1)$

1.3. LJ decomposition (3)

- **Property** of *repeated decompositions*:

  If a decomposition $\{R_1, \ldots, Ri, \ldots, R_m\}$ of R has the lossless-join property w.r.t. a set of FDs F on R, and a decomposition $\{Q_1, \ldots, Q_k\}$ of R_i has the lossless join property w.r.t. the projection of F on R_i, then the decomposition $\{R_1, \ldots, R_{i-1}, Q_1, \ldots, Q_k, R_{i+1}, \ldots, R_m\}$ of R has the lossless-join property w.r.t. F.

- We will use these properties to test whether a decomposition is lossless-join.
- If a relation is decomposed into more than two relations, an efficient algorithm is available to test whether the decomposition is lossless-join (this is the well-known *Chase* algorithm).

1.3. LJ decomposition (4)

- Example:

Let R = (A, B, C), and F = {A $\rightarrow$ B, B $\rightarrow$ C}

The decomposition D = {AB, BC} is LJ because
(AB $\cap$ BC) $\rightarrow$ (BC – AB) is in F^+ (it is even in F).

Clearly this decomposition is DP:
A $\rightarrow$ B is in $\Pi_{AB}(F)$ and B $\rightarrow$ C is in $\Pi_{BC}(F)$.

The decomposition D = {AC, BC} is not LJ because
(AC $\cap$ BC) $\rightarrow$ (BC – AC) is not in F+, and
(AC $\cap$ BC) $\rightarrow$ (AC – BC) is not in F+.

One can see that this decomposition is not DP.

1.4. Decomposition algorithms (1)

- It is always possible to find a *DP decomposition* w.r.t. to a set of FDs into 3NF relations.
- The following algorithm guarantees *only the DP* property; it does *not* guarantee the *LJ property*.
- **Algorithm (**relational synthesis**)**
  Input: relation R and set of FDs F on R
  Output: a DP decomposition D of R into 3NF relations.

1. Find a minimal cover G for F (use the relevant algorithm);
2. For each LHS X of a FD in G, create a relation schema $XA_1 \ldots A_n$ in D, where $X \rightarrow A_1, \ldots, X \rightarrow A_n$ are the only FDs in G with X as LHS (X is the key of this relation);
3. Place any remaining attributes (that have not been placed in any relation) in a single relation schema to ensure the attribute preservation property;

1.4. Decomposition algorithms (2)

- It is possible to find a *LJ decomposition* w.r.t. to a set of FDs *into BCNF relations*.
- The following algorithm *guarantees only the LJ* property; it does *not* guarantee the *DP property.*
- **Algorithm**
  Input: relation R and set of FDs F on R
  Output: an LJ decomposition D of R into BCNF relations.

  1. Set D := {R};
  2. While there is a relation schema Q in D that is not in BCNF do
     {
     Choose a relation schema Q in D that is not in BCNF;
     find a FD $X \rightarrow Y$ in F on Q that violates BCNF;
     replace Q in D by two relation schemas $(Q - Y)$ and $(X \cup Y)$;
     };

1.4. Decomposition algorithms (3)

- It is *not* always possible to find *a LJ and DP decomposition into BCNF relations*.

  If we want a decomposition that has the *DP* *<u>and</u>* *the LJ properties* we have to be satisfied with relation schemas in *3NF* rather than in BCNF.

1.4. Decomposition algorithms (4)

- The following algorithm guarantees *both the DP and LJ properties*.

- **Algorithm (**relational synthesis**)**
  Input: relation R and set of FDs F on R
  Output: a LJ and DP decomposition D of R into 3NF relations.

1. Find a minimal cover G for F (use the relevant algorithm);
2. For each LHS X of a FD in G, create a relation schema $XA_1 \ldots A_n$ in D, where $X \rightarrow A_1, \ldots, X \rightarrow A_n$ are the only FDs in G with X as LHS (X is the key of this relation);
3. If none of the relation schemas in D contains a key, then create one more relation schema in D that contains attributes that form a key of R;

2. Other dependencies (1)

- Consider a relation R that contains all the attributes in a database. Let $X, Y \subseteq R$ and $Z = R - XY$

  A **Multivalued Dependency (MVD)**, denoted $X \rightarrow\rightarrow Y$, in R specifies the following *constraint* on any relation instance r(R):
  If two tuples t_1, t_2 exist in r such that $t_1[X] = t_2[X]$, then two tuples t_3, t_4, should also exist in r with the following properties:

  $t_3[X] = t_4[X] = t_1[X] = t_2[X]$
  $t_3[Y] = t_1[Y]$ and $t_4[Y] = t_2[Y]$
  $t_3[Z] = t_2[Z]$ and $t_4[Z] = t_1[Z]$

- We can specify *sound and complete inference rules* for MVDs, as we did for FDs.

- A **fourth normal form** has been defined based on MVDs

2. Other dependencies (2)

- Consider the lists of attributes X and Y of relation schemas R and S, respectively which have the same number of attributes. Assume that the domain for each pair of corresponding attributes (one from X and one from Y) are compatible.

- An **inclusion dependency** R.X < S.Y specifies the constraint that for any instance r of R and any instance s of S, $\Pi_X(r) \subseteq \Pi_Y(s)$.

- Example

  DEPARTMENT.MGRSSN < EMPLOYEE.SSN
  PROJECT.DNUM < DEPARTMENT.DNUMBER

  These constraints can also be expressed as referential integrity constraints. However, an inclusion dependency is more general than a referential integrity constraint. The following inclusion dependency cannot be expressed as a ref. integrity constraint.

  PROJECT.PNUMBER < WORKS_ON.PNO